ZedBoard Matrix-Multiplication Accelerator Full Implementation and Data-Movement Guide

PS/PL architecture • AXI interfaces • DDR/DMA • weight-stationary mapping • tiling • padding • validation

Target: Digilent ZedBoard, Zynq-7000 XC7Z020-CLG484

0. Scope and important status

This is a candidate implementation guide, not proof that a complete 16×16 PE design fits. The example uses signed INT8 inputs and INT32 accumulation. Final dataflow, precision, bus widths, clock, PE implementation and buffers must be verified in RTL and Vivado.

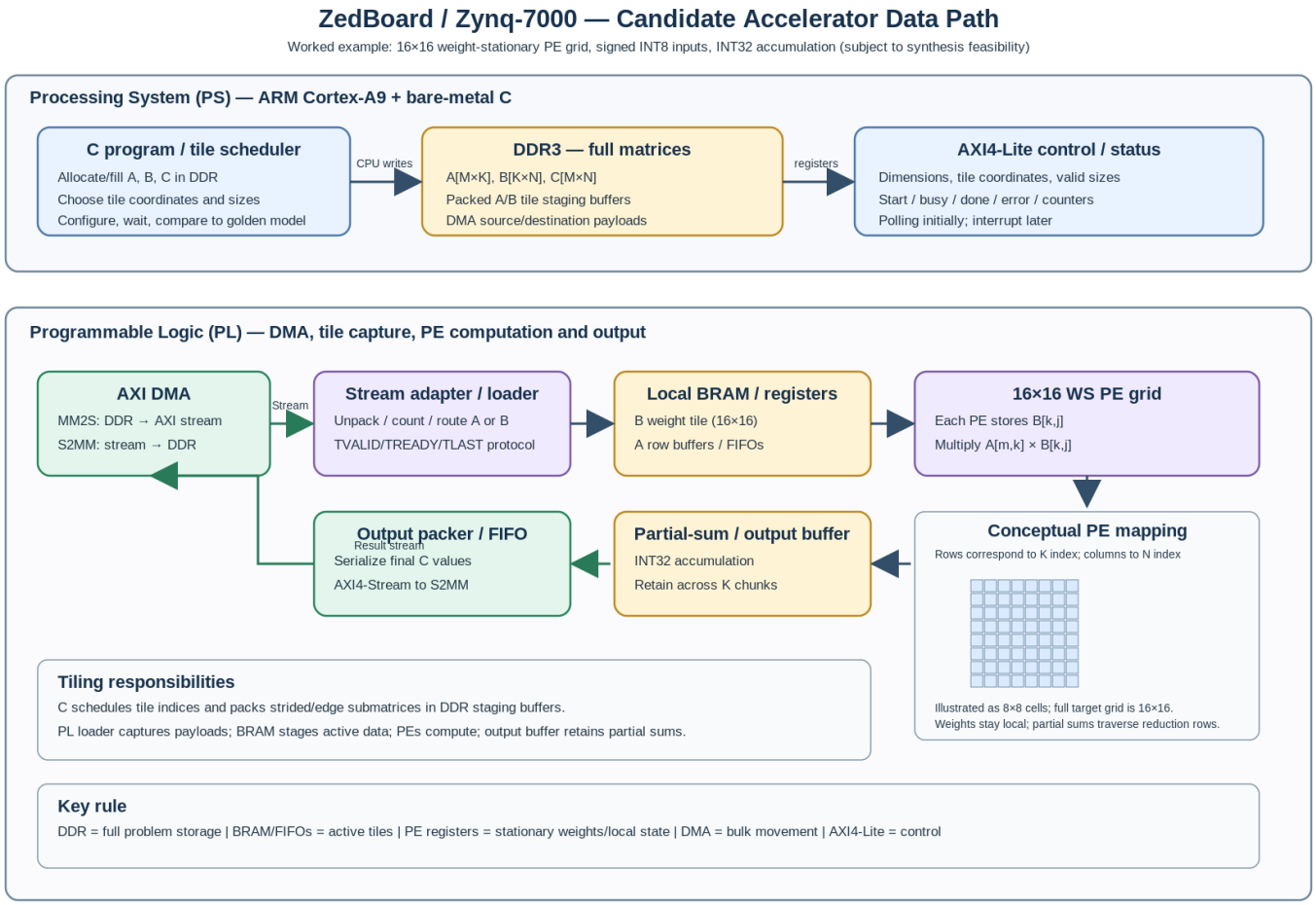
Fixed facts	Candidate choices	Must verify
ZedBoard XC7Z020; 512 MB DDR3; 140 BRAM36; 220 DSP; 53,200 LUTs	Bare-metal C; AXI DMA; AXI4-Lite control; AXI4-Stream payload; WS grid	256-PE mapping, resources, timing, stream packing and tile buffers

A 16×16 grid has 256 PEs. If every PE uses one DSP, it exceeds the 220 DSP inventory by 36. LUT mapping or a different implementation might be possible, but it increases resource/timing risk. Compare against smaller arrays.

1. Board constraints

Resource	Fact	Implication
PL logic	53,200 LUTs; 106,400 FFs	All control, PE, FIFO and adapter logic share this fabric.
DSP	220 slices	256 PE design is not one-DSP-per-PE feasible.
BRAM	140×36-Kbit; nominal ~630 KiB	Actual usable tile memory is lower due to other uses and port/banking fragmentation.
DDR3	512 MB, 32-bit interface on PS memory controller	Full matrices live in DDR; capacity is not guaranteed bandwidth.
PS/PL	ARM Cortex-A9; GP/HP/ACP paths	CPU orchestrates; DMA accesses DDR through a configured HP path.
Clocks	PS 33.333 MHz and PL 100 MHz board oscillators	100 MHz oscillator does not guarantee accelerator fmax.

2. System architecture



DDR holds full A, B and C. C schedules tiles. AXI4-Lite configures control/status. AXI DMA MM2S reads DDR payloads and emits AXI4-Stream. PL loader fills local buffers. The PE array computes. Output packer + S2MM writes results back to DDR.

3. AXI roles

Interface	Role
AXI4-Lite	Small register reads/writes; dimensions, mode, start, status. Not bulk matrix payload.
AXI4-MM	Addressed burst path used by DMA for DDR reads/writes.
AXI4-Stream	Payload sequence. Define order, operand phases, byte packing and TLAST convention.
PS HP port	Provides PL AXI master access to PS DDR/OCM; not a DMA engine itself.
AXI DMA	Moves data between memory-mapped and stream interfaces; does not understand GEMM tiling by itself.

A stream transfer occurs when TVALID and TREADY are both high. If TREADY is low, the source holds its payload/control stable. For the first design, simple/direct-register DMA avoids scatter-gather complexity. Program address and byte count; follow the PG021 version matching the configured Vivado IP.

4. Worked 16×16 matrix multiplication

A[16×16] INT8 × B[16×16] INT8 = C[16×16] INT32
C[i,j] = Σ(k=0…15) A[i,k] × B[k,j]

Object	Shape	Raw bytes
A	16×16 INT8	256
B	16×16 INT8	256
C	16×16 INT32	1,024
Total	—	1,536 B = 1.5 KiB

One candidate WS mapping: PE grid row = reduction index k, column = output column j. PE[k,j] stores B[k,j]. For each output row i, A[i,k] is presented to row k; each PE multiplies A[i,k] by its stored B[k,j], and partial sums are combined along the reduction direction. After all 16 terms, output C[i,0…15] is emitted. Exact skew, forwarding and pipeline timing must be designed and verified in RTL.

Phase	Data/action
1. Preload B	Load 256 INT8 weights into PE-local registers (256 bytes before protocol overhead).
2. Process each i=0…15	Feed one A row (16 bytes) with required skew; reuse B weights; accumulate 16 terms per C element in INT32.
3. Drain C	Emit 256 INT32 values = 1,024 bytes.
4. Store/check	S2MM writes C to DDR; C invalidates cache as needed and compares with software golden model.

5. End-to-end transfer sequence

• Reserve DDR buffers and fill A/B row-major; reserve C.

• Clean/flush CPU cache lines for DMA source buffers if data cache is enabled; follow BSP alignment/API rules.

• Program accelerator dimensions, precision and tile/phase registers.

• Arm S2MM destination and expected output length before data can be emitted.

• Start MM2S for B preload; PL loader captures bytes and loads stationary weight registers. Wait for a documented load-complete handshake.

• Start MM2S for A rows/activation payload; loader applies the agreed order/skew and feeds the PE grid.

• PEs compute in PL; output/partial-sum buffer retains results as required.

• Output packer sends 1,024 bytes of C as INT32 values; S2MM writes DDR.

• Wait for DMA and accelerator done/error; invalidate C cache lines before CPU reads if data cache is enabled.

• Compare every C[i,j] with a software golden model and report mismatches.

For larger row-major matrices, subtiles may not be contiguous. A simple first implementation packs each A/B tile into contiguous DDR staging buffers in C. Later, consider 2-D DMA/scatter-gather or a PL address generator if supported by the chosen configuration.

6. General tiling

For A[M×K], B[K×N], C[M×N], choose Mt, Kt and Nt. A tile is Mt×Kt; B tile is Kt×Nt; C tile is Mt×Nt. For each output tile, initialize C_tile once, then accumulate across K chunks: C_tile += A_tile × B_tile. Never reset C_tile at the start of every K chunk.

Conceptual CPU loop:

```
for im in range(0,M,Mt):
for jn in range(0,N,Nt):
clear C_tile once
for ik in range(0,K,Kt):
compute valid_M/N/K
pack A_tile and B_tile
pad edge inputs if needed
DMA/load, compute and accumulate
write only valid C_tile values to DDR
```

Tiling has multiple owners: C chooses tile indices and packs data; DMA moves bytes; the PL loader places data into local storage; the PE array computes; the PL output buffer retains partial sums; DMA writes final output tiles.

7. Rectangular dimensions and padding

Rectangular matrices are normal. Only K dimensions must match. Padding is optional and is usually applied to edge-tile payloads, not the entire full matrix. Example: A is 19×27, B is 27×13, C is 19×13, with Mt=16, Kt=16, Nt=8.

Edge tile	Nominal	Valid	Padding/handling
A last M,K	16×16	3×11	Pad 13 rows + 5 K columns with zero.
B last K,N	16×8	11×5	Pad 5 K rows + 3 columns with zero.
C last M,N	16×8	3×5	Only write valid 3×5 outputs; mask/discard the rest.

K splits into 16+11; pad the last 11-wide K chunk with five zeros if the hardware requires Kt=16. INT8 zero is 0x00. Do not write padded C values outside the true matrix. Carry valid_M/N/K or use a fixed padded payload plus masks.

Row-major addresses: A[r,c]=base_A+(r·K+c)×sizeof(A); B[r,c]=base_B+(r·N+c)×sizeof(B); C[r,c]=base_C+(r·N+c)×sizeof(C). Define strides consistently in bytes or elements.

8. Tile storage and maximum

For INT8 A/B and INT32 partial sums, Sraw ≈ Mt×Kt + Kt×Nt + 4×Mt×Nt bytes. Double-buffer A/B and single-buffer C: Sraw ≈ 2MtKt + 2KtNt + 4MtNt. Include FIFOs, weight registers, metadata, alignment and BRAM banking overhead.

Equal T	Single A+B+C	A+B double, C single	All double
16	1.5 KiB	2 KiB	3 KiB
32	6 KiB	8 KiB	12 KiB
64	24 KiB	32 KiB	48 KiB
128	96 KiB	128 KiB	192 KiB
256	384 KiB	512 KiB	768 KiB

Nominal BRAM aggregate is 630 KiB, but it is not all available for tile data. Exact maximum tile size cannot be claimed before buffer design and Vivado reports. Separate PE grid size, output tile size, local buffer capacity, and full matrix size in DDR.

9. Vivado implementation order

• Configure Zynq PS, PL clock and one HP port; verify DDR/clock settings.

• Add AXI DMA MM2S/S2MM in simple/direct mode; connect memory masters through HP and control through GP/AXI-Lite.

• Create accelerator IP: AXI-Lite registers, AXI4-Stream input/output, loader, buffers, PE array, output packer.

• Validate register read/write, then DMA loopback DDR→stream→DDR.

• Test stream input capture and fixed output pattern before arithmetic.

• Verify one PE and a small array against the software reference.

• Implement one tile, then K-chunk accumulation, then rectangular edge handling.

• Add ping-pong buffers/interrupts/wider streams only after correctness and measurement.

10. Verification checklist

Test	Purpose
2×2 / 4×4 known values	Basic arithmetic and ordering
16×16 worked tile	Weight preload, activation flow, output drain
Random signed inputs	Negative products and corner cases
19×13 output with K=27	Rectangular edges and padding/masks
Multiple K chunks	Partial sums persist
TREADY stalls	No dropped or duplicated stream beats
Repeated runs with changed data	Cache coherency and stale-buffer issues
Synthesis/implementation reports	LUT/FF/DSP/BRAM, timing slack, achieved clock

11. References

• [1] Digilent ZedBoard Hardware User Guide v2.2: https://files.digilent.com/resources/programmable-logic/zedboard/ZedBoard_HW_UG_v2_2.pdf

• [2] AMD Zynq-7000 TRM UG585, PS–PL AXI: https://docs.amd.com/r/en-US/ug585-zynq-7000-SoC-TRM/PS-PL-AXI-Interfaces

• [3] AMD DS190, Zynq-7000 resource overview: https://www.amd.com/content/dam/xilinx/support/documents/data_sheets/ds190-Zynq-7000-Overview.pdf

• [4] AMD AXI DMA PG021 Core Overview: https://docs.amd.com/r/en-US/pg021_axi_dma/Core-Overview

• [5] AMD AXI DMA PG021 Direct Register Mode: https://docs.amd.com/r/en-US/pg021_axi_dma/Direct-Register-Mode-Simple-DMA

• [6] AMD AXI DMA PG021 2-D Transfers (configuration/mode dependent): https://docs.amd.com/r/en-US/pg021_axi_dma/2-D-Transfers

• [7] AMD AXI overview: https://www.amd.com/en/products/adaptive-socs-and-fpgas/intellectual-property/axi.html

Bottom line: Full matrices stay in DDR; C schedules/optionally packs tiles; AXI4-Lite controls; DMA + AXI4-Stream moves payloads; local BRAM/registers stage data; the PE array computes; S2MM writes completed output tiles back to DDR. Padding is for edge tiles only when useful. The 16×16 PE example is a worked mapping, not a proven fit: 256 PEs exceed 220 DSPs if one DSP per PE.